Reflective Essays – Course Portfolio

Martin Demel

Department of Science, Technology, Engineering & Math, Houston Community College

6263-ITAI-4370-AI 5/6G Comm & ORAN Net-S10-14071

Tawanda Chiyangwa

July 23rd, 2026.

Essay 1: How My Understanding of Telecommunications Evolved

I came into this course from the operations side of technology. My background is IT service management and customer success: ITIL processes, platforms, incident bridges. In that world the network was always a layer below me, something other teams ran, something I escalated to, a box on an architecture diagram labeled "connectivity." I could talk about networks fluently at the service level and not at all one layer down. If you had asked me in early June what actually happens between a phone and a website, I would have given you an answer about routers and protocols that sounded fine and explained nothing.

The first shift came in week one, and it was smaller and more concrete than I expected. Assignment 1 asked for definitions, such as what a telecommunications system is and what a transmitter, channel, and receiver do, and I could have answered those from reading alone. What

changed my understanding was Laboratory 1, where I had to build the chain of source, modulator, channel, demodulator, and receiver in about thirty lines of NumPy. Watching eight random bits get multiplied onto a cosine carrier, buried in Gaussian noise, and then recovered perfectly by nothing more than multiplying by the same carrier and averaging was the moment telecommunications stopped being vocabulary and became mechanism. The five-block diagram I drew in Draw.io was no longer notation; every arrow on it was a line of code I had written and could break on purpose.

The second shift was learning that noise is the entire story. Before this course I thought of noise as an imperfection at the edges of communication. The BER work in Lab 1 reframed it: the channel is the only block that damages the signal, and every other block in the receiver exists to undo that damage. When my 1,000,000-bit Monte-Carlo simulation landed on the theoretical BER curve, with 7.78 x 10^-4 simulated against 7.73 x 10^-4 predicted at 7 dB, I understood for the first time why engineers fight over single decibels: in the steep region of that curve, one extra dB roughly halves the error rate. Communication engineering is the discipline of managing a known enemy with mathematically predictable behavior.

The third shift connected physics to maps. The free-space path loss lab (Lab 2) is four lines of math, but computing that a 28 GHz mmWave signal loses about 30 dB more than a 900 MHz signal at the same kilometer, roughly one-thousandth of the power, explained something I had passively observed for years: why 5G coverage maps look the way they do, and why mmWave lives on lamp posts in dense downtowns while low-band covers entire counties. I stopped seeing network topology as a planning choice and started seeing it as propagation physics made visible.

The fourth shift was the one that landed closest to my professional home. Studying the 5G core for Assignment 2, I expected faster hardware. What I found was that the EPC's dedicated boxes had been dissolved into cloud-native software services on a common bus, and that the Network Repository Function, which lets those services register and discover each other at run time, is exactly the service-discovery pattern I know from microservices platforms and ITIL service catalogs. That was the week telecommunications stopped feeling like a foreign country. The 5G core is a distributed software system, network slicing is multi-tenancy, and MEC is a placement decision under a latency budget. I already had the mental furniture; the course showed me it applied.

By the final weeks, the evolution completed itself: the network became something that learns. Traffic prediction (Lab 3), model comparison (Lab 4), self-organizing simulations (midterm), and edge model compression

(Lab 5) each added a piece of the picture in which the RAN is operated by control loops, with rApps planning on slow timescales and xApps acting on fast ones, fed by exactly the kind of forecasting models I was building each week.

If I compress the whole journey into one sentence: I started the course seeing telecommunications as infrastructure that carries information, and I finished seeing it as a layered discipline in which physics constrains architecture, architecture enables software, and software now hosts intelligence, with each layer's constraints explaining the shape of the one above it. The bits, the decibels, and the microservices are one subject, and for the first time I can move between those layers instead of standing outside all of them.

Essay 2: How I Applied AI Concepts to Solve Telecommunications Problems

Every AI application I built in this course attacked the same underlying telecommunications problem from a different angle: a network has finite resources and variable demand, and the gap between the two is where congestion, latency, and outages live. Over five weeks I applied machine learning to anticipate that demand, simulation to understand its dynamics, and model compression to put the intelligence where the network needs it. This essay walks through the four problems I solved and what each taught me.

Problem 1: Anticipating demand before it arrives

The first real application was Laboratory 3. The problem statement was operational: an hour from now, how much traffic will this network carry? I generated a synthetic year of hourly traffic with daily, weekly, and business-hours structure, engineered lag features (the traffic 1 hour and 24 hours earlier, plus a 7-day rolling average), and trained a Random Forest. Two decisions mattered more than the model choice. First, splitting the data in time order rather than shuffling, so the model was evaluated the way a deployed forecaster would be: on hours it had never seen. Second, reading the feature importances rather than just the score: the previous hour dominated at 0.816, which told me network traffic is habitual, and that most of the predictive power in a RAN scheduler comes from very recent history. The final test R^2 of 0.893 was less important to me than understanding why it was achievable.

Problem 2: Choosing the right model, and catching a lie

Laboratory 4 turned one predictor into a controlled comparison: ARIMA(2,1,2), linear regression on sixteen engineered features, and a PyTorch LSTM, all forecasting the same simulated year. The results were extreme (ARIMA at R^2 of negative 0.181, regression at 0.572, the LSTM at 0.897), and each number taught a different lesson. A non-seasonal ARIMA forecasting a long horizon flattens to the mean of cyclical traffic; hand-built lag features can recover the weekly rhythm for a linear model; a sequence model learns the daily shape directly from raw data.

But the most valuable thing that happened in this lab was a bug. The course brief's feature code computed moving averages over windows that included the hour being predicted. The model scored a perfect R^2 of 1.0, and it was completely worthless, because it was reading the answer out of its own input. Recognizing that as target leakage, and fixing it by shifting every window to use only past hours, was the single most transferable AI skill I practiced all term: a suspiciously good score is a bug report, not a result. In a real O-RAN deployment, a leaky traffic model would validate beautifully and then fail in production, steering resources with predictions it can no longer make.

Problem 3: Networks that react to themselves

The midterm applied AI-adjacent modeling to the control side. I built a SimPy discrete-event simulation of packets queueing through three routers, which made latency tangible as an emergent property of shared resources. I built a Mesa agent-based model of five routers on a small-world graph, each rerouting traffic when its load crossed 80%, so congestion appeared and dissolved from purely local rules with no central controller. And I built a lag-feature regression predicting traffic so that routing decisions could be made before saturation instead of after. Together the three pieces are a miniature of a Self-Organizing Network: observe, predict, act. The Mesa model in particular changed how I think about "self-healing." It is not one algorithm but a property that emerges when every node follows simple adaptive rules.

Problem 4: Fitting the intelligence where it has to live

The final application inverted the question: instead of making models smarter, make them smaller. Multi-access Edge Computing only reduces latency if the model can physically run at the edge, so in Laboratory 5 I compressed a 96.2%-accurate baseline classifier three ways: pruning half the weights (96.7% after fine-tuning, twice as small), INT8 quantization (96.1%, four times smaller), and knowledge distillation into a

student one-tenth the size (92.7%, and the only variant that actually ran faster, at 0.029 ms, because zeroed weights and simulated integers shrink storage rather than compute on standard hardware). When the brief's TensorFlow code crashed in my environment, I reimplemented the entire lab in PyTorch, which forced me to understand each technique well enough to build it rather than run it, including implementing the distillation loss the brief had defined but never actually used.

What the four problems add up to

The through-line I take away is that AI in telecommunications is not a chatbot bolted onto a network. It is an unglamorous, operational loop (forecast demand, allocate resources, detect anomalies, compress the model until it fits next to the radio) running on the RIC's two timescales. Every week of this course I built one piece of that loop with my own hands, and the pieces compose: the forecasting of Labs 3 and 4 is what an xApp consumes, the emergent adaptation of the midterm is what SON automates, and the compression of Lab 5 is what makes any of it deployable at the edge. That composition, more than any single model, is what I now understand.

Essay 3: Skills I Improved and Areas for Future Growth

Skills I improved

Scientific Python as a working language. I started the course able to run Python; I ended it able to investigate with it. Across five labs and a midterm I worked the full stack: NumPy and SciPy for signal simulation, pandas for time-indexed data, Matplotlib for every figure in this portfolio, scikit-learn for classical models, statsmodels for ARIMA and seasonal decomposition, SimPy and Mesa for simulation, and PyTorch for neural networks. The clearest single marker of growth: in week one I extended a provided BPSK script; by week five I rebuilt an entire TensorFlow lab in PyTorch from scratch because the original crashed in my environment.

Evaluation discipline. The most important skill I sharpened has no library: knowing when a result can be trusted. This course drilled it repeatedly. Time-ordered train/test splits score forecasters on the future rather than a shuffled past. Validating a simulation against closed-form theory, as when my Monte-Carlo BER landed on the theoretical curve, is the test that the code is right. Comparison caveats must be stated honestly; ARIMA forecast a whole horizon while the LSTM predicted one hour ahead, so the gap reflects both model and task. Above all, catching

the target-leakage bug in Lab 4's brief, where a moving-average feature that included the target produced a perfect, meaningless R^2 of 1.0, taught me to treat a too-good metric as the start of an investigation.

Reproducibility habits. Every notebook in this portfolio runs top-to-bottom with a fixed random seed and its outputs embedded, so any result I cite (0.893, 121.4 dB, 7.78 x 10^-4) can be regenerated by anyone who clones the repository. That habit came directly from having to cross-reference my own numbers between labs, assignments, and this portfolio.

Neural-network fundamentals by construction. Building an LSTM forecaster, then pruning, quantizing, and distilling an MLP, taught me these techniques at the level of tensors rather than API calls: what a hidden state carries, what an L1 pruning mask actually zeroes, why INT8 needs a scale factor per layer, and why a distillation loss softens the teacher's logits with a temperature. Implementing the distillation loss the course brief had defined but never used was the moment when "reading about a technique" and "having the technique" became different things for me.

Communicating technical work. Four APA papers, six documented notebooks, and this GitHub portfolio, all structured, indexed, and written for a reader who was not in the room, pushed my technical writing from notes for myself toward documentation for others. Coming from IT service management, I already valued operational documentation; this course taught me to apply the same standard to analytical work: state the problem, show the method, report the number, interpret it, admit the caveat.

Areas for future growth

Real data. Every dataset I modeled this term was synthetic: traffic I generated myself, with patterns I chose. Real network telemetry is messier, with missing intervals, sensor drift, and regime changes no sine wave predicts. My next step is applying this term's pipeline to live captures (Wireshark traces and public telecom datasets) where the ground truth was not written by me.

Radio beyond free space. FSPL is the physics of a perfect world. Real link budgets involve fading, multipath, shadowing, and interference, the phenomena the Module 2 slides introduced but the labs did not simulate. I want to model Rayleigh and Rician channels and see how the clean BER curves of Lab 1 degrade.

Hands-on O-RAN. I can now explain the Near-RT and Non-RT RIC split and what an xApp consumes; I have not yet deployed one. Working with an open RIC platform (such as the O-RAN Software Community stack)

and packaging my Lab 3 forecaster as an actual xApp is the most direct continuation of this course.

Edge deployment on physical hardware. My edge lab simulated a constrained device by disabling the GPU. The honest next step is exporting the distilled model to a Raspberry Pi or microcontroller (ONNX or TFLite) and measuring real latency and power, where the storage-versus-compute distinction I discovered in Lab 5 becomes physical.

Uncertainty and depth in forecasting. My models predict a number; operators need a range. Probabilistic forecasting (prediction intervals, quantile loss) and stronger sequence architectures are where I want to push the time-series work next.

Closing

The skill I am most conscious of gaining is not on either list: it is the confidence to open an unfamiliar technical layer (modulation, propagation, core architecture, RAN intelligence) and expect to find something learnable rather than something opaque. That expectation, plus the evaluation discipline to distrust easy results, is what I carry forward from this course into whatever I operate, analyze, or build next.